Centro Universitário de Excelência
Sistemas de Informação

MecâniQA – Padrões de Projeto, DTOs e Diagrama de Atividade

Autores: Andrey Ramos Ferraz
Arthur Prates
Lucas Peixoto Soares Araújo
Rhuan Cassius Brito Silva

Vitoria da Conquista, 2026

Agenda

Esta apresentação aborda a evolução da API da MecâniQA: as novas entidades e regras de negócio, o Padrão Builder, os DTOs e Mappers, e o Diagrama de Atividade do endpoint de criação de OS.

1. Item 1:
Contexto e Objetivos da Sprint

2. Item:
Novas Entidades e Regras de Negócio

3. Item:
Padrão de Projeto Builder

4. Item:
DTOs e Mappers

5. Item:
Diagrama de Atividade (Endpoint)

6. Item:
Considerações Finais e Referências

Contexto e Objetivos

O que foi desenvolvido nesta etapa?

- Evolução do back-end Java + Spring Boot.
- Persistência em memória (padrão Singleton estático).
- Foco em padrões estruturais e criacionais.
- Proteção de dados com DTOs.
- Proibição do uso de Lombok (construtores, getters e setters manuais).

Novas Entidades e Regras de Negócio

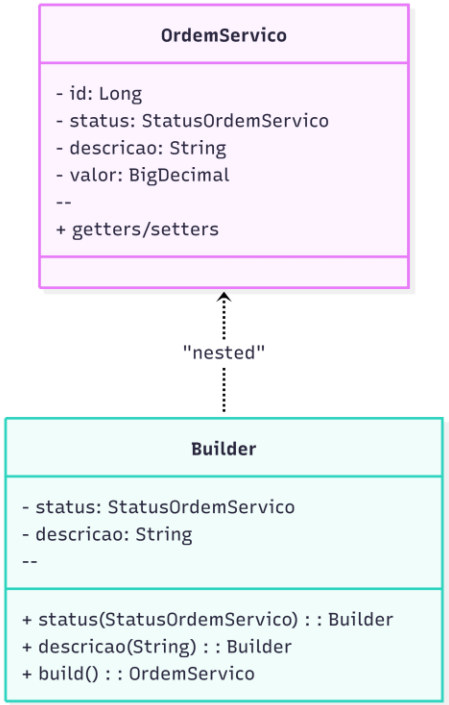
Itens para discussão:

- **Ordem de Serviço (OS):** Status obrigatórios: *Aberto, Pendente de Pagamento, Pago, Em Execução, Executado*.
- **Pedido de Peças:** Status obrigatórios: *Orçando, Pendente de Pagamento, Pago/Faturado, Entregue*.
- **Entidade Associativa:** Relaciona Peças e Pedidos; Permite múltiplas peças em um mesmo pedido com suas respectivas quantidades.

Implementação do Builder na OS

Itens para discussão:

- **Problema:** Complexidade de instanciar uma Ordem de Serviço com muitos atributos.
- **Solução:** Padrão Builder para construção passo a passo.
- **Benefícios:** Código mais limpo, legível e seguro na criação de objetos complexos.



DTOs e Mappers

Itens para discussão:

- **O que são DTOs?** (Data Transfer Objects).
- **Por que usar?** A API não recebe ou devolve entidades de domínio puras.
- **Mappers:** Conversão de DTOs para as classes de modelo existentes.
- **Vantagem:** Maior segurança e controle sobre os dados trafegados.

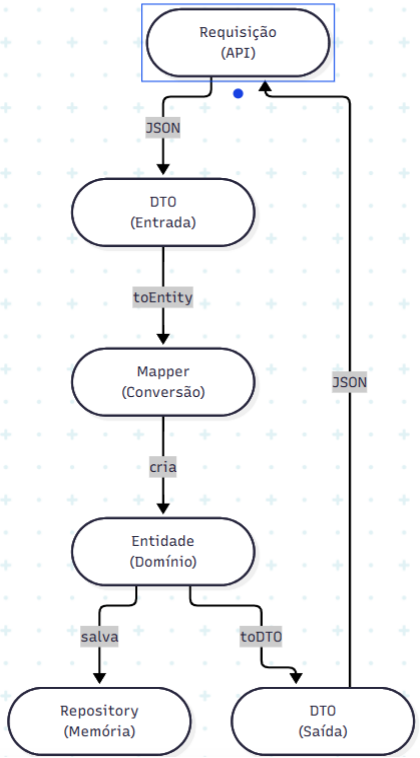
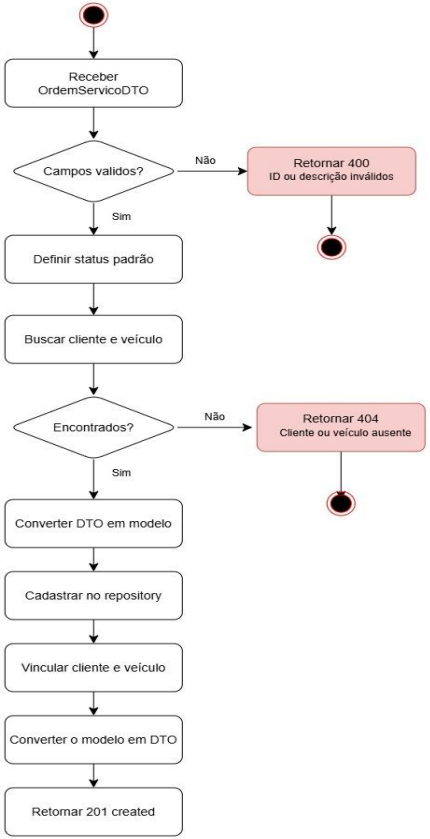


Diagrama de Atividade



Considerações finais

Pontos de interesse:

- A API está mais robusta e preparada para processos complexos.
- O uso de Builder e DTOs facilitou a manutenção e segurança.
- Aprendizado prático sobre padrões de projeto e separação de responsabilidades.

Referências

- SPRING. Spring Boot Documentation. [S.l.]: Spring, [s.d.]. Disponível em: https://docs.spring.io/spring-boot/.

- REFACTORING.GURU. Builder. [S.l.]: Refactoring.Guru, 2024. Disponível em: https://refactoring.guru/design-patterns/builder.

- PAVLOV, D. Generic DTO Assembler - Getting started. [S.l.]: Inspire Software, 2013. Disponível em: https://www.inspire-software.com/confluence/display/GeDA/Getting+started.

- CAVALLOTTI, J. A. L. jDTO Binder 1.4 User's Guide. [S.l.: s.n.], 2013. Disponível em: https://github.com/jDTOBinder/jDTO-Binder/raw/master/book/jdto.pdf.